AIAC-13.3

Name:G.Harika

Ht.no:2303A51612

Batch.no:05

Task 1: Refactoring – Removing Code Duplication

Prompt

Refactor the following legacy Python code to remove repeated logic.
Create reusable functions to calculate the area and perimeter of rectangles.
Ensure the code has no duplicated blocks and include proper docstrings for each function.

Objective

To eliminate repeated logic by extracting reusable functions.

Task Description

Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.

Starter (Legacy) Code

```
# Legacy script with repeated logic

print("Area of Rectangle:", 5 * 10)

print("Perimeter of Rectangle:", 2 * (5 + 10))

print("Area of Rectangle:", 7 * 12)

print("Perimeter of Rectangle:", 2 * (7 + 12))

print("Area of Rectangle:", 10 * 15)

print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Expected Outcome

- A reusable function to calculate area and perimeter
- No duplicated code blocks
- Proper docstrings for all functions

Code:

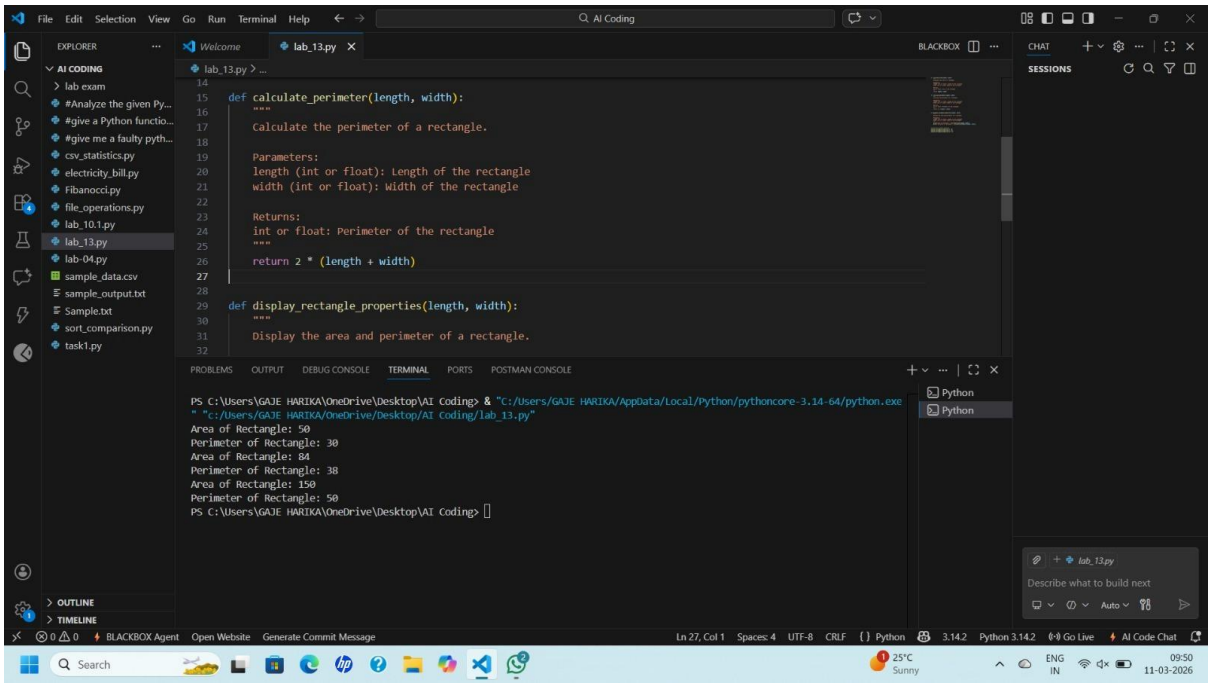
```
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.
```

```
    Parameters:
    length (int or float): Length of the rectangle
    width (int or float): Width of the rectangle

    Returns:
    int or float: Area of the rectangle
    """
    return length * width
def calculate_perimeter(length, width):
    """
    Calculate the perimeter of a rectangle.
    Parameters:
    length (int or float): Length of the rectangle
    width (int or float): Width of the rectangle
    Returns:
    int or float: Perimeter of the rectangle
    """
    return 2 * (length + width)
def display_rectangle_properties(length, width):
    """
    Display the area and perimeter of a rectangle.
    Parameters:
    length (int or float): Length of the rectangle
    width (int or float): Width of the rectangle
    """
    print("Area of Rectangle:", calculate_area(length, width))
    print("Perimeter of Rectangle:", calculate_perimeter(length, width))
# Using the reusable function
display_rectangle_properties(5, 10)
```

display_rectangle_properties(7, 12)

display_rectangle_properties(10, 15)

Output:



Explanation

1. calculate_area(length, width) function calculates the **area of a rectangle** using the formula length × width.
2. calculate_perimeter(length, width) function calculates the **perimeter of a rectangle** using the formula 2 × (length + width).
3. display_rectangle_properties(length, width) function **calls the above two functions** and prints the area and perimeter.
4. The function is called three times with different values (5,10), (7,12), and (10,15).
5. This refactored code **removes repeated calculations and uses reusable functions**, making the code cleaner and easier to maintain.

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements

using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:

found = False

for n in names:

if s == n:

found = True

if found:

print(f"{s} is in the list")

else:

print(f"{s} is not in the list")
```

Expected Outcome

• Optimized solution using set lookups or comprehensions

• Performance comparison before and after refactoring

Prompt:

Refactor the following Python code to improve performance.
The code currently uses **nested loops to check if elements exist in a list**.
Optimize the program using **set lookup or list comprehension** to reduce time complexity and improve efficiency.

Code:

```
names = ["Alice", "Bob", "Charlie", "David"]

search_names = ["Charlie", "Eve", "Bob"]


# Convert list to set for faster searching

names_set = set(names)


for s in search_names:

    if s in names_set:
```

```
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Output:

```
AIAC_project.py    AIAC-13.3.py ×
AIAC-13.3.py > s
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]

# Convert list to set for faster searching
names_set = set(names)

for s in search_names:
    if s in names_set:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   POSTMAN CONSOLE

PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING> & C:/Users/vempa/App
ctures/Desktop/HCL LAB/AI ASSISTANT CODING/AIAC-13.3.py"
Charlie is in the list
Eve is not in the list
Bob is in the list
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL LAB\AI ASSISTANT CODING>
```

Explanation

1. names list stores the available names.
2. search_names list contains names to be checked.
3. set(names) converts the list into a **set for faster lookup**.
4. The for loop goes through each name in search_names.
5. if s in names_set checks whether the name exists in the set.
6. If the name exists, it prints **"is in the list"**, otherwise **"is not in the list."**

Task 3: Refactoring – Extracting Reusable Functions

Objective

To modularize code by extracting calculations into reusable functions.

Task Description

Refactor a legacy script where price and tax calculations are written inline.

Starter (Legacy) Code

```
price = 250

tax = price * 0.18

total = price + tax

print("Total Price:", total)

price = 500

tax = price * 0.18

total = price + tax

print("Total Price:", total)
```

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

Prompt:

Refactor the following Python legacy code to remove repeated calculations.
Create a reusable function **calculate_total(price)** that calculates the tax (18%) and the total price.
Ensure the code is **clean, reusable, and includes proper docstrings**.

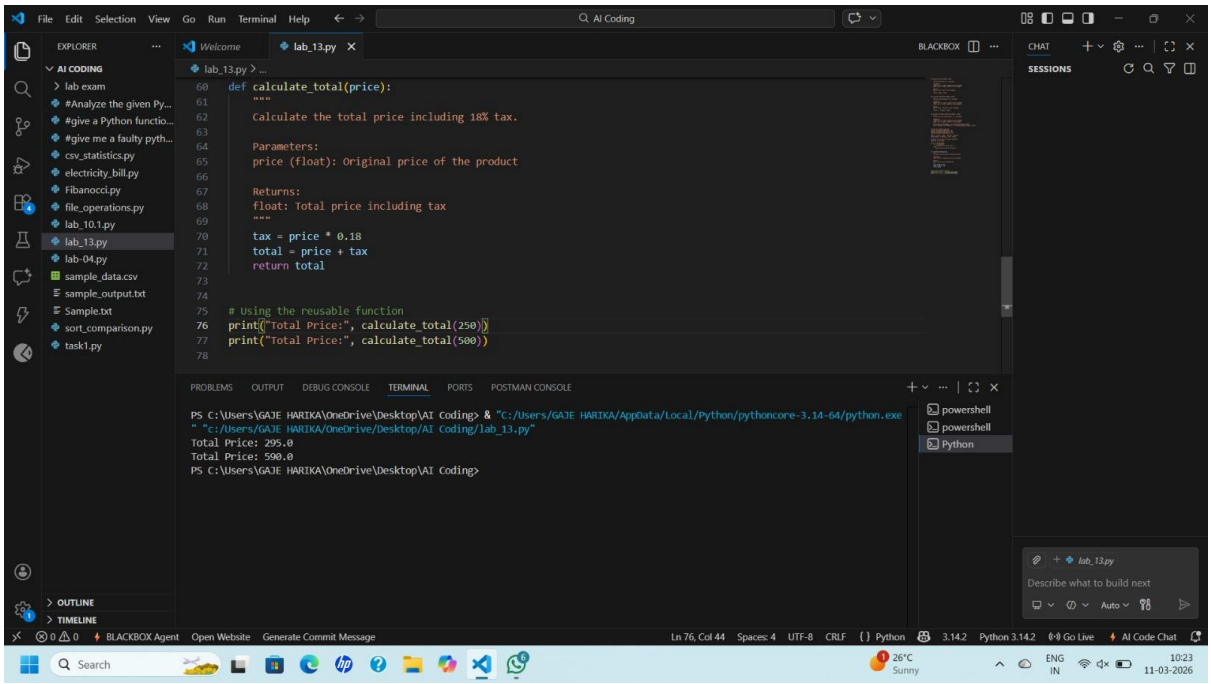
Code:

```
def calculate_total(price):
    """
    Calculate the total price including 18% tax.
    Parameters:
    price (float): Original price of the product
    Returns:
    float: Total price including tax
    """
    tax = price * 0.18
    total = price + tax
    return total
# Using the reusable function
```

print("Total Price:", calculate_total(250))

print("Total Price:", calculate_total(500))

Output:



Explanation:

1. calculate_total(price) function is created to calculate tax and total price.
2. The docstring explains the function, parameters, and return value.
3. Tax is calculated using the formula:

$$tax = price \times 0.18$$

4. Total price is calculated as:

$$total = price + tax$$

5. The function returns the total price.
6. The function is called with different values 250 and 500 to avoid repeating code.

Task 4: Refactoring – Replacing Hardcoded Values with Constants

Objective

To improve maintainability by replacing magic numbers with named constants.

Task Description

Use AI to identify hardcoded values and replace them with constants.

Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))

print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

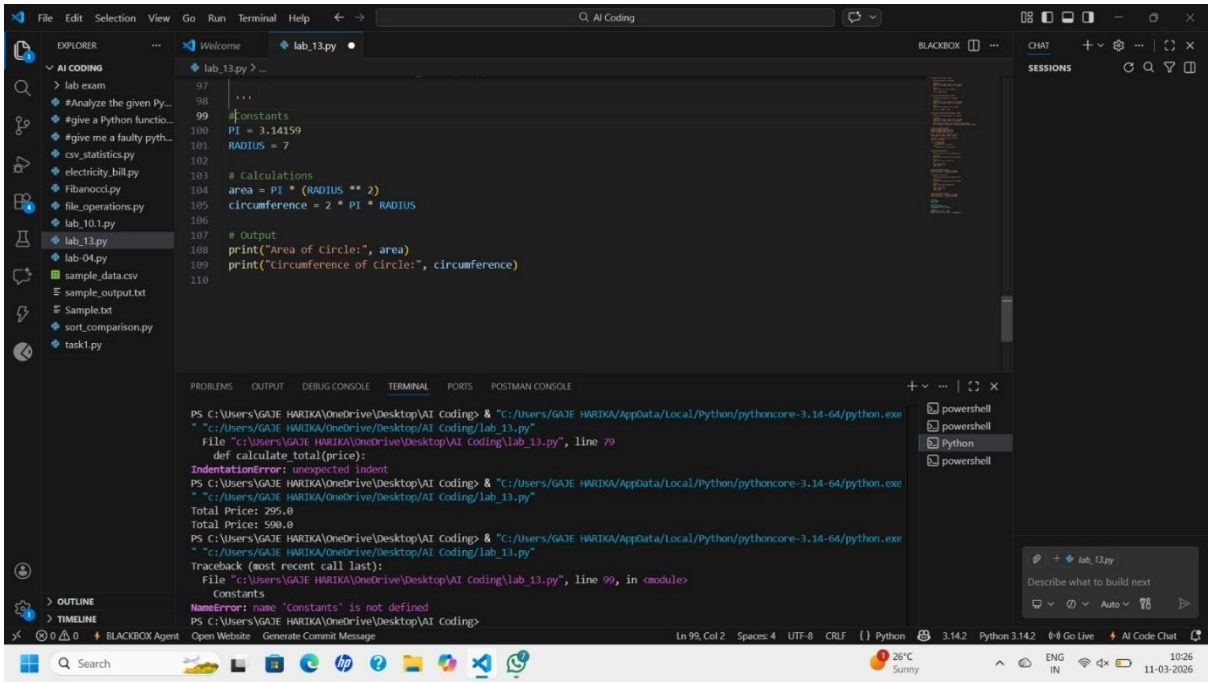
Prompt:

Refactor the following Python code by replacing hardcoded values (magic numbers) with named constants.
Define constants such as **PI** and **RADIUS** to improve code readability and maintainability.

Code:

```
# Constants

PI = 3.14159

RADIUS = 7

# Calculations

area = PI * (RADIUS ** 2)

circumference = 2 * PI * RADIUS

# Output

print("Area of Circle:", area)

print("Circumference of Circle:", circumference)
```

Output:



Explanation:

1. PI and RADIUS are defined as constants to replace hardcoded values.
2. area = PI * (RADIUS ** 2) calculates the area of the circle using the formula:

$$Area = \pi r^2$$

3. circumference = 2 * PI * RADIUS calculates the circumference of the circle using the formula:

$$Circumference = 2\pi r$$

4. The results are printed using the print() function.

Task 5: Refactoring – Improving Variable Naming and Readability

Objective

To enhance readability using descriptive variable names and comments.

Task Description

Refactor a script with unclear variable names.

Starter (Legacy) Code

a = 10

b = 20

c = a * b / 2

print(c)

Expected Outcome

• Descriptive variable names

• Inline comments explaining logic

• Identical output

Prompt:

Refactor the following Python code to improve **variable naming and readability**.
Replace unclear variable names with **descriptive names** and add **inline comments explaining the logic**.
Ensure the output remains the same.

Code:

Base and height values

base = 10

height = 20

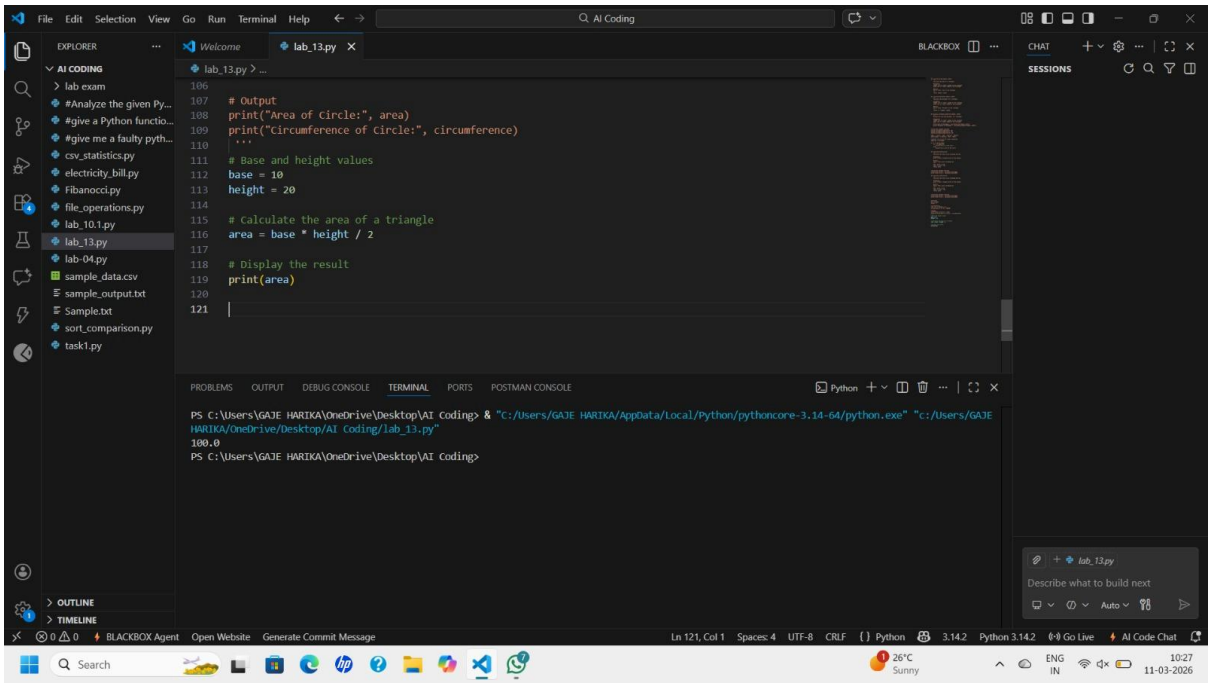
Calculate the area of a triangle

area = base * height / 2

Display the result

print(area)

Output:



Explanation:

1. Variable a is renamed to base for clarity.
2. Variable b is renamed to height to describe its purpose.
3. Variable c is renamed to area since it stores the triangle's area.
4. Inline comments explain each step of the calculation.
5. The program calculates the area of a triangle using the formula:

$$Area = \frac{base \times height}{2}$$